

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

01 List as Stacks

02

03

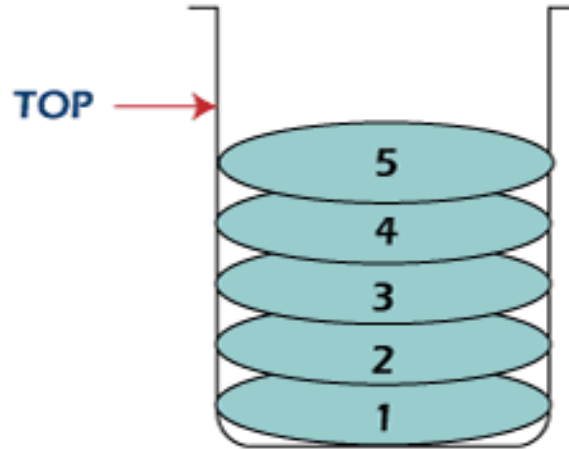
04

05

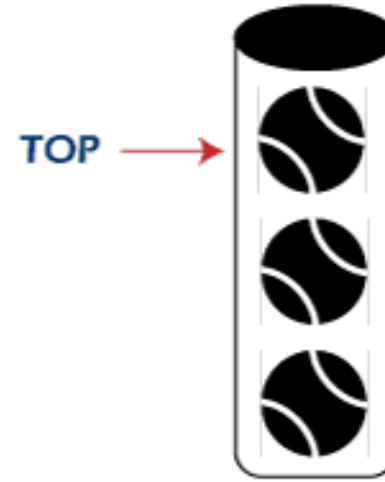
Stack



Stack of Coins



Stack of Plates



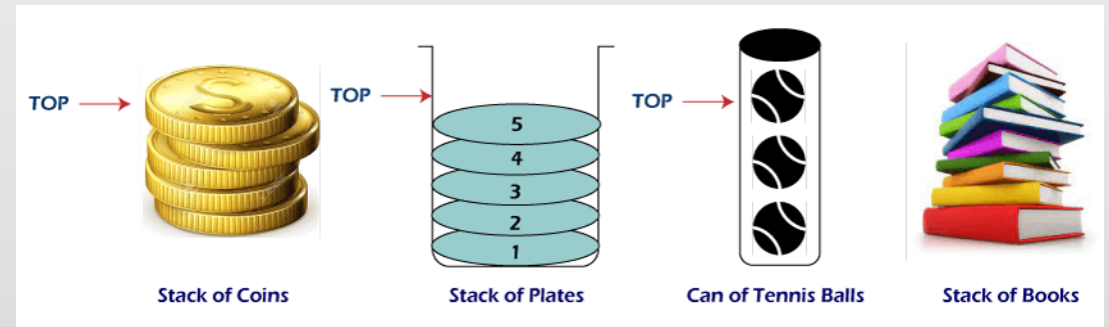
Can of Tennis Balls



Stack of Books

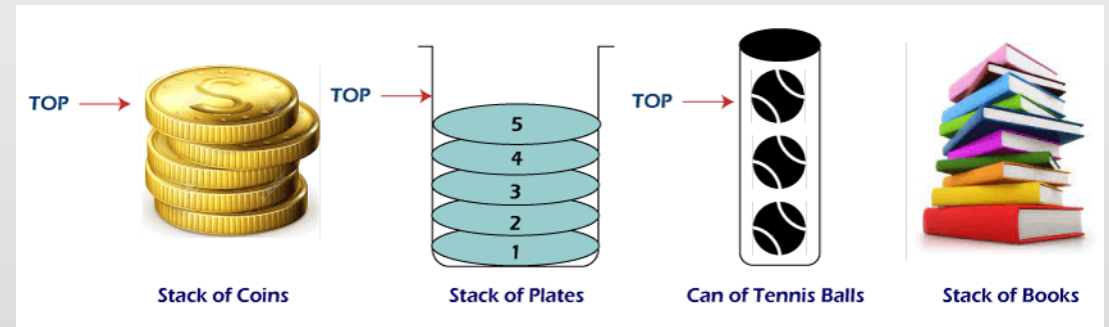
Stack

- Stack is an important data structure
- It stores its elements in an ordered manner
- To understand the concept of a stack see the analogy in picture



Stack

- See the pile of plates, stack of books
- Now, when you want to remove a plate, you remove the topmost plate first



Stack

- Hence you can add or remove an element of a stack only at the top of the stack
- A stack is a linear data structure
- It is called a Last-In-First-Out (LIFO) data structure
- i.e. the element which was inserted last is the first one to be taken out

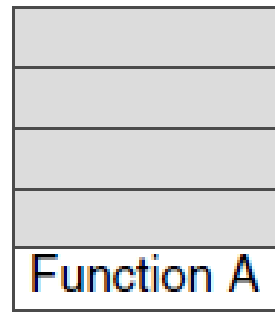
Stack

- Where do we need stacks in Computer science?
- In function calls:
 - Consider that we are executing a function A
 - In the course of its execution, function A calls function B
 - Function B calls another function C
 - Function C calls function D

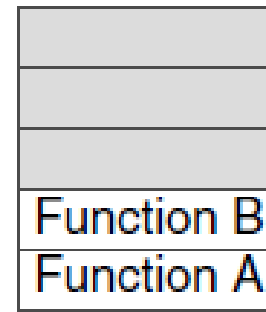
Stack

- To keep track of the returning point of each active function, a special stack called system stack or call stack is used
- Whenever a function calls another function, the calling function is pushed onto the top of the stack
- This is because after the called function is executed, the control is passed back to the calling function

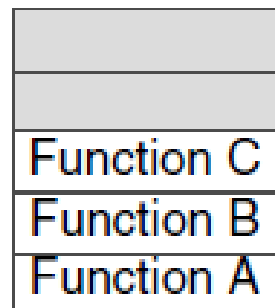
Stack



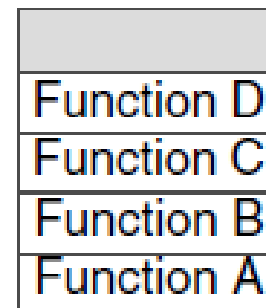
When A calls B, A is pushed on top of the system stack. When the execution of B is complete, the system control will remove A from the stack and continue with its execution.



When B calls C, B is pushed on top of the system stack. When the execution of C is complete, the system control will remove B from the stack and continue with its execution.



When C calls D, C is pushed on top of the system stack. When the execution of D is complete, the system control will remove C from the stack and continue with its execution.



When D calls E, D is pushed on top of the system stack. When the execution of E is complete, the system control will remove D from the stack and continue with its execution.

Stack

function is executed. This is shown in Figure 8.3. The system stack ensures a proper execution order of functions. Therefore, stacks are frequently used in situations where the order of processing is very important, especially when the processing needs to be postponed until other conditions are fulfilled.

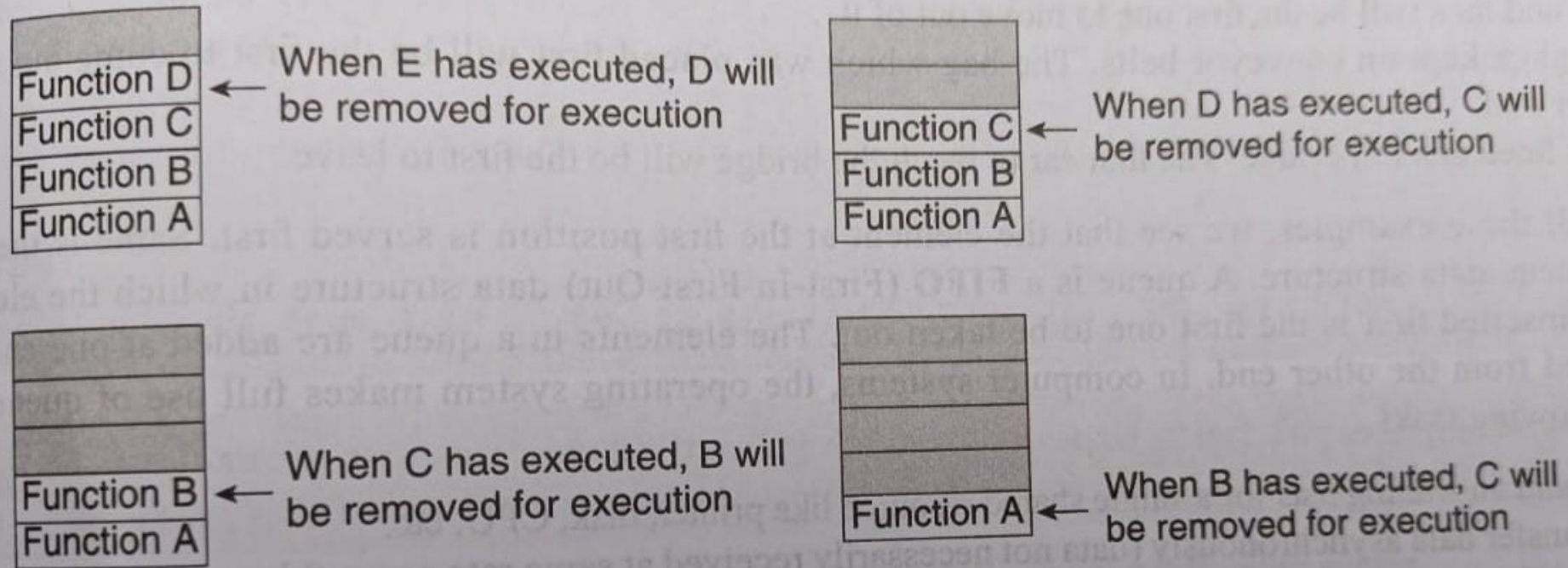


Figure 8.3 Returning from called functions

Stack

- A stack supports three basic operations: **push**, **pop**, and **peep** (or peek)
- push operation adds an element at the top of the stack

Stack

- pop operation removes the element from the top of the stack
- peep operation returns the value of the element at the top of the stack (without deleting it)

Using Lists as Stack

- In Python, the list methods make it very easy to use a list as a stack
- Ex, to push an element in the stack, we use the `append()` method

Using Lists as Stack

- In Python, the list methods make it very easy to use a list as a stack
- Ex, to pop an element from the stack, we use the `pop()` method

Using Lists as Stack

- In Python, the list methods make it very easy to use a list as a stack
- Ex, to perform pop operation use the **slicing** operation

Using Lists as Stack

```
stack = [1,2,3,4,5,6]
print("Original stack is : ", stack)
stack.append(7)
print("Stack after push operation is : ", stack)
stack.pop()
print("Stack after pop operation is : ", stack)
last_element_index = len(stack) - 1
print("Value obtained after peep operation is : ",
      stack[last_element_index])
```

OUTPUT

```
Original stack is : [1, 2, 3, 4, 5, 6]
Stack after push operation is : [1, 2, 3, 4, 5, 6, 7]
Stack after pop operation is : [1, 2, 3, 4, 5, 6]
Value obtained after peep operation is : 6
```


Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025